

DTS ZSC Coordination Dashboard

Dynamic task sequencing for zero shot human AI coordination in a simulated warehouse workflow

Purpose

Demonstrate adaptive task ordering and worker allocation under changing human behavior.

Current state

Working research simulator, trained checkpoint, dashboard, evaluations, tests, and shareable archive.

Verified scope

One to three orders and the dashboard team configurations documented in this guide.

Important boundary

This is a research prototype, not a production warehouse control system.

DOCUMENT MAP

What this guide covers

This document explains the complete project, the work delivered during the latest revision, how a simulation step is processed, how to operate and share the system, when it can fail, and which capabilities still require future engineering.

1

Executive overview

Purpose, audience, scope, and current status.

2

Delivered capabilities

Dashboard, policies, observer, metrics, and reliability work.

3

System architecture

Components, data flow, and API boundaries.

4

Coordination logic

Orders, dependencies, priorities, workers, and actions.

5

Human Observer

Behavior signals and enhanced policy input.

6

Dashboard operation

Controls, panels, interpretation, and typical workflow.

7

Evidence and testing

Saved evaluation results and automated verification.

8

Installation and sharing

Setup, startup, shutdown, and archive contents.

9

Failure guide

Symptoms, causes, and recovery actions.

10

Current limitations

What is intentionally outside the implemented scope.

11

Recommended roadmap

Practical path from prototype to production.

12

Technical reference

Files, endpoints, metrics, and handoff checklist.

How to read performance claims

Live dashboard runs illustrate behavior. Research cards use saved evaluation artifacts. Neither should be interpreted as evidence of real warehouse safety, productivity, or employee performance without a separate real world validation program.

1. EXECUTIVE OVERVIEW

A research simulator for adaptive coordination

The project studies how an AI coordinator can choose the next warehouse task and assign it to an AI or simulated human partner while orders arrive, priorities differ, deadlines tighten, and human behavior changes.

6

Tasks per order

Two fetch tasks followed by packing, labeling, weighing, and dispatch.

10

Human profiles

Five training profiles and five completely held out evaluation profiles.

4

Policy choices

PPO, Greedy, Static, and Random coordinators.

6

Priority presets

Random, urgent, medium, routine, urgent first, and urgent later.

41

Automated tests

Environment, model, statistics, dashboard, and scenario coverage.

3

Dashboard workloads

One, two, or three concurrent orders.

Primary users

User	What the system provides	What it does not provide
Researcher	Repeatable simulated episodes, baselines, partner splits, metrics, ablations, and saved charts.	Evidence about real workers or physical warehouses.
Reviewer or client	A live visual explanation of priorities, task dependencies, allocation, and policy behavior.	A production service level agreement or safety certification.
Developer	Modular Python environment, policy implementations, Flask API, tests, checkpoints, and result files.	A deployment platform, database, authentication layer, or hardware adapter.

Current delivery status

The source project runs locally, the dashboard responds on port 5000, the complete test suite passes, and a Windows compatible share archive has been created and extraction tested.

2. DELIVERED CAPABILITIES

What has been built and improved

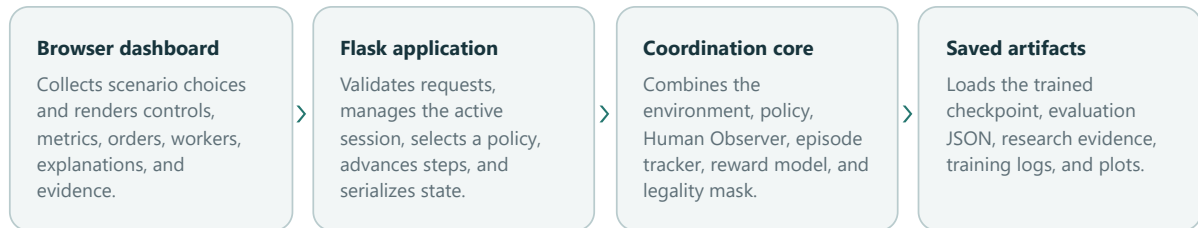
Capability	Delivered behavior	Where it appears
Order priority control	Six selectable priority scenarios. Every order displays a numeric priority, urgency class, deadline, slack, and priority rank.	Header controls and order cards
Continuous flow	Workers continue progressing on active tasks during wait steps. Three order scenarios receive a 90 step budget.	Environment and progress panels
Human Observer output	The five exact observer features sent to the coordinator are visible for every human.	Human Observer panel
AI performance	AI utilization, progress, tasks completed, steps per task, throughput, completed work share, and idle time are reported.	AI worker cards
Sequencing basis	Active work, ready candidates, priority, slack, difficulty, and dependency value are displayed with a policy specific explanation.	Step sequencing panel
Decision explanations	Each decision records source, operational reason, role logic, and observed human signal.	Coordinator explanation panel
Policy safety	Illegal assignments are blocked by a canonical action mask. Reassignment cost reflects discarded progress.	Environment and metrics
PPO compatibility handling	If the selected scenario does not match the trained PPO dimensions, a visible Greedy fallback is used. An untrained model is never shown as PPO.	Notice and status bar
Random baseline correction	Random selection chooses among fresh ready assignments and no longer destroys active progress through repeated takeovers.	Random policy
Reliable playback	Step and Run calls cannot overlap, stale reset responses are ignored, and API failures are shown clearly.	Dashboard controls
Professional presentation	Visible status text, titles, labels, and metrics use consistent language and readable punctuation.	Entire dashboard
Shareable distribution	A native Windows ZIP excludes machine specific files and has passed a full extraction test.	Desktop archive

Transparency rule

An operational candidate ranking is shown for inspection, but it is not presented as a reconstruction of hidden PPO logits. Fallback and continuity guard decisions are explicitly labeled.

3. SYSTEM ARCHITECTURE

How the components work together



One simulation step

- 1 Read current state**
The coordinator receives the enhanced observation and the environment information dictionary, including the legal action mask.
- 2 Select a legal action**
The policy chooses a task and worker pair or WAIT. The environment owns legality so policy and simulator cannot disagree.
- 3 Apply the assignment**
A pending eligible task starts, an allowed active task may be reassigned, or current work is preserved.
- 4 Advance every worker**
Human and AI workers progress simultaneously. Completed tasks unlock their dependents.
- 5 Score the outcome**
The environment applies task rewards, deadline effects, parallel work bonuses, avoidable idle penalties, and reassignment cost.
- 6 Update observer and metrics**
Human behavior history, episode counters, CES inputs, logs, and explanations are updated.
- 7 Render state**
The API returns JSON and the browser refreshes all live panels without reloading the page.

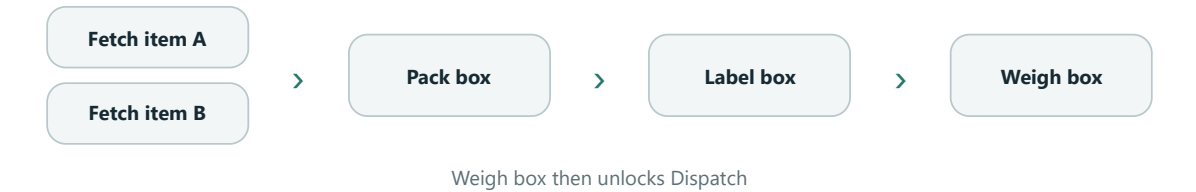
API surface

Route	Method	Purpose
/	GET	Returns the complete self contained dashboard.
/api/config	GET	Returns profiles, policies, priorities, defaults, and saved research summaries.
/api/reset	POST	Validates a scenario and starts a fresh in memory episode.
/api/step	POST	Advances one or more bounded simulation ticks.
/api/state	GET	Returns the current episode without advancing it.

4. COORDINATION LOGIC

Orders, dependencies, priorities, and actions

Order dependency graph



Task	Difficulty	Baseline human time	AI time	Prerequisite
Fetch item A	0.3	4.0 steps	3.5 steps	None
Fetch item B	0.4	5.0 steps	4.0 steps	None
Pack box	0.6	8.0 steps	5.5 steps	Both fetch tasks
Label box	0.2	2.0 steps	2.0 steps	Pack box
Weigh box	0.2	2.0 steps	1.5 steps	Label box
Dispatch	0.3	3.0 steps	2.5 steps	Weigh box

Priority and deadline behavior

Order priority ranges from 0.20 to 1.00. Higher priority produces a tighter deadline. Orders arrive five steps apart, so the coordinator may need to switch attention when later work becomes urgent.

Preset	Values	Intended demonstration
Random	Sampled from 0.20 to 1.00	Natural scenario variation
All urgent	0.95	Tight deadline pressure
All medium	0.55	Balanced pressure
All routine	0.20	Generous slack
Urgent first	0.95, 0.20	Front loaded urgency
Urgent later	0.20, 0.95	Priority shift after a later arrival

Why the dashboard can appear quiet after step two

Both fetch tasks may already be active. The ready queue is then empty because packing requires both fetch tasks to finish. WAIT is correct in that state, and active worker progress continues on every tick.

4. COORDINATION LOGIC CONTINUED

State, actions, reward, and policy choices

Observation and action dimensions

Block	Features	Examples
Each task	6	Status, difficulty, assignee, order priority, deadline slack, arrival state
Each worker	6	Busy, waiting, progress, observed speed, observed error rate, fatigue
Global state	3	Elapsed time, task completion fraction, order completion fraction
Each human observer	5	Smoothed behavior and completion features

For the trained default scenario with two orders, one AI, and one human, the policy receives 92 inputs and selects from 25 actions. There are 24 task and worker assignments plus WAIT.

Reward design

Event	Effect	Reason
Every step	Minus 0.02	Encourages timely completion.
Task completion	10 times difficulty	Rewards useful progress.
Order on time	20 times 1 plus priority	Values urgent deliveries more highly.
Order late	8	Retains value while reducing the completion bonus.
Past deadline	Minus 0.25 times priority per step	Applies continuing lateness pressure.
Avoidably idle worker	Minus 0.30 per worker per step	Encourages use of available capacity only when eligible work exists.
Parallel work	Plus 0.40 per extra busy worker	Rewards synchronized progress.
Illegal action	Minus 1.0	Protects valid workflow state.
Reassignment	Minus 2.0 times discarded progress	Discourages disruptive late takeovers.
All orders complete	Plus 25	Rewards solving the complete episode.

Policy behavior

Policy	Behavior
PPO	Uses a trained neural policy with masked actions. The saved model supports the default two order, one AI, one human shape.
Greedy	Ranks ready tasks using priority, slack, and dependency value, then selects the fastest estimated free worker.
Static	Uses fixed task order and rotating worker assignment. It does not adapt to observed human behavior.
Random	Selects a random legal fresh assignment. It waits when no free worker and ready task pair exists.

5. HUMAN OBSERVER

How human variability becomes policy input

The policy is not given a profile label. It observes behavior through worker state and a rolling feature block. This keeps the held out partner test meaningful inside the simulator.

Simulated behavior axes

Axis	Effect in the simulator
Speed	Changes the time required to complete a task.
Error rate	May add redo time to the current task.
Fatigue	Accumulates during work and slows later progress.
Response latency	Creates a delay between assignment and work start.
Speed jitter	Adds task level pace variation.
Strategy	Changes relative pace on easier and harder work.

Five observer features per human

Feature	Meaning
Smoothed speed	Rolling mean of observed pace over the recent behavior window.
Smoothed error rate	Rolling mean of observed errors.
Completion ratio	Observed completions divided by observed assignments.
Speed trend	Difference between the newer and older halves of recent pace history.
Completed workload	Observed human completions divided by total tasks in the scenario.
Dashboard transparency	The exact values are displayed with sample and assignment counts.

Training and held out profiles

Training profiles	Held out profiles	Separation rule
average, fast, slow, expert, novice	tired, easy first, hard first, hesitant, erratic	The two sets are asserted to be disjoint in code.

Interpretation boundary

These profiles are programmed simulations, not psychological or occupational assessments. Observer values should not be used to evaluate real employees.

6. DASHBOARD OPERATION

How to run and interpret a live episode

- 1 Select a policy.** Choose PPO, Greedy, Static, or Random. A selection change resets the episode.
- 2 Select a partner.** Training and held out profiles are grouped and labeled.
- 3 Select workload and team.** Choose one to three orders and one of the available team compositions.
- 4 Select order priority.** Use routine, urgent, mixed, or random conditions.
- 5 Inspect the initial state.** Check order priorities, deadlines, ready fetch tasks, workers, and the sequencing explanation.
- 6 Advance the episode.** Step advances one tick, Step times five advances five ticks, and Run continues until paused or complete.
- 7 Interpret progress.** Task nodes show ready, blocked, active, responding, or complete. Inner bars show task execution progress.
- 8 Review the outcome.** The completion status reports CES, reward, tasks, orders, and whether the step limit was reached.

Panel reference

Panel	Question answered
Metric cards	How much reward, coordination efficiency, progress, synchronization, and conflict has occurred?
Order cards	Which order is most important, what is ready, and what is blocked?
Worker cards	Who is active, how much progress is made, and how is each worker performing?
Allocation	How many assignments went to humans and AI, and how often did roles switch?
Human Observer	What behavior summary is supplied to the coordinator?
Sequencing basis	What constraints and operational ranking shape the next step?
Decision explanations	What was selected, from which source, and why was it operationally reasonable?
Event log	What assignments, completions, waits, reassignments, and terminal events occurred?
Research evidence	What do saved, repeated evaluations report?

Healthy blocked state

An empty ready queue does not necessarily indicate a failure. If active work is shown, the environment is advancing it until a dependency becomes available.

7. EVIDENCE AND TESTING

What the saved evaluation demonstrates

The bundled evaluation uses 50 episodes for each of ten partner profiles. That produces 500 simulated episodes per policy in the saved comparison.

Policy	Overall CES	Solve rate	Interpretation
PPO	0.690	99.8%	Best saved overall CES and solve rate.
Greedy	0.665	91.2%	Strong adaptive scripted baseline.
Static	0.482	76.0%	Fixed workflow comparison.
Random	0.444	56.2%	Random legal assignment comparison from the saved evaluation.

0.686
Seen partner CES
Mean PPO CES for the five training profiles.

0.693
Held out partner CES
Mean PPO CES for five profiles not used in training.

99.6%
Held out solve rate
Saved result across 250 held out profile episodes.

CES = 0.35 completion + 0.20 speed + 0.15 synchronization + 0.20 on time rate - 0.10 conflict rate

The separate idle analysis reports a fully idle rate of 12.49% for PPO and 18.16% for Static across the saved study, which corresponds to 31.2% lower fully idle time for PPO.

Automated verification

- All 41 tests pass in the project test runner.
- Legality mask and environment action checks are compared across visited states.
- Checkpoint save, load, configuration metadata, and dimension guards are tested.
- Training and held out partner separation is tested.
- Observer ablation and blind partner behavior are tested.
- Dashboard API payloads, visible panels, invalid inputs, and selector combinations are tested.
- Every exposed baseline, workload, and team combination completes in the dashboard matrix test.

Evidence caveat
The dashboard research cards describe stored evaluation results. Changing the live selector does not recompute those research statistics.

8. INSTALLATION AND SHARING

How to install, run, stop, and share

Windows setup from the shared ZIP

```
1. Extract human_rl_july_share.zip
2. Open PowerShell in the extracted human_rl_july folder
3. python -m venv .venv
4. .\.venv\Scripts\Activate.ps1
5. pip install -r requirements.txt
6. python app.py
7. Open http://127.0.0.1:5000
```

macOS or Linux setup

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python app.py
```

System requirements

Requirement	Purpose	Notes
Python	Runs the simulator, model, evaluation, and Flask server.	A modern supported Python 3 release is recommended.
PyTorch	Loads and executes the PPO network.	CPU operation is supported for the dashboard.
Gymnasium and NumPy	Environment and numerical state.	Installed from requirements.txt.
Flask	Local web application and API.	The bundled server is for development use.
Pandas and Matplotlib	Evaluation analysis and saved plots.	Not required for every live dashboard action, but included in project dependencies.

Share archive

The validated archive is named `human_rl_july_share.zip`. It includes source, tests, documentation, trained checkpoints, ablation checkpoints, logs, results, and images. It excludes the local virtual environment, Git history, Python caches, and server logs.

Extraction validation
The corrected archive was created with the native Windows ZIP writer and successfully extracted with Windows Expand Archive. Required files were checked after extraction.

Stopping the server

If the server is running in a visible terminal, press Ctrl C. If it was started in the background, stop the Python process listening on port 5000 before starting another copy.

9. FAILURE GUIDE

When the project can break and how to recover

Symptom	Likely cause	Recovery
Browser cannot connect	The Flask server is not running, crashed, or started in a different folder.	Activate the environment, run <code>python app.py</code> , and confirm port 5000.
Port 5000 is already in use	A previous server instance or another application owns the port.	Stop the existing listener or change the Flask port in <code>app.py</code> .
Import error at startup	Dependencies were not installed in the active Python environment.	Run <code>pip install -r requirements.txt</code> and verify the active interpreter.
PPO fallback notice	The selected order or team shape does not match the trained checkpoint, or the checkpoint is missing.	This is controlled behavior. Use two orders with one AI and one human for PPO, or retrain for the desired shape.
Research cards unavailable	<code>evaluation.json</code> or <code>research_evidence.json</code> is missing, malformed, or incompatible.	Restore the bundled result files or rerun evaluation. Live simulation can still operate.
No ready tasks after early steps	Both fetch tasks are active and later work is dependency blocked.	Inspect active worker progress and continue stepping. This is usually expected.
Step limit reached	A slow or erratic partner, static ordering, errors, or high workload consumed the available ticks.	Review the event log and worker performance. Reset with another policy or scenario.
Dashboard displays old wording	The browser cached an earlier page.	Use Ctrl F5 or clear the site cache.
ZIP does not extract	An obsolete archive created with the earlier packaging method was used.	Use the corrected 29.1 MB native Windows ZIP with the validated hash.
One user's reset changes another view	The server has a single process global session.	Use one user at a time or implement isolated persistent sessions.
State disappears after restart	Episode state is stored only in memory.	Expected behavior. Add a database or serialized session store for persistence.
Model load is slow	PyTorch initialization and checkpoint loading can take several seconds on first use.	Wait for the request to finish and avoid repeated reset clicks.

Do not expose the bundled Flask server directly to the internet

It has no authentication, tenant isolation, rate limiting, production process manager, transport security configuration, or hardened deployment settings.

10. CURRENT LIMITATIONS

What is not possible in the current version

The items below are not impossible in principle. They are simply not implemented or validated in the current prototype.

Capability	Current status	What would be required
Production deployment	Not implemented No hardened server, authentication, database, monitoring, or operational support model.	Production WSGI service, reverse proxy, TLS, secrets, access control, logging, monitoring, backups, and deployment automation.
Independent multiuser sessions	Not implemented All users share one in memory episode.	Session identifiers, persistent state, concurrency control, and tenant isolation.
PPO for arbitrary team and workload shapes	Limited The saved PPO checkpoint supports two orders, one AI, and one human.	Separate checkpoints per shape or a variable size architecture followed by retraining and evaluation.
Real warehouse control	Not implemented No connection to robots, scanners, warehouse systems, sensors, or safety controllers.	Hardware adapters, event ingestion, command protocols, safety interlocks, simulation to reality validation, and operational certification.
Real human behavior inference	Not validated All people are simulated profiles.	Ethically approved data collection, privacy controls, bias assessment, consent, calibration, and field studies.
Guaranteed optimal schedule	Not guaranteed PPO and heuristics choose actions but do not prove global optimality.	An exact optimization formulation, bounded problem size, solver integration, and optimality analysis.
Causal AI explanations	Limited Current explanations summarize observable inputs and operational logic.	Validated attribution methods, counterfactual tests, and explanation quality studies.
Live training in the dashboard	Not implemented The UI loads a saved checkpoint.	Job management, progress streaming, resource limits, checkpoint registry, cancellation, and result tracking.
Persistent audit history	Not implemented Only the current in memory log is shown.	Database schema, immutable event records, export, retention, and privacy policy.
Real world performance claims	Not supported Evidence is entirely simulated.	Pilot deployment, controlled comparison, safety review, statistical plan, and independent validation.

Most important limitation

The system demonstrates a coordination method in a controlled simulator. It must not make autonomous decisions about real workers or equipment without substantial new engineering, governance, and validation.

11. RECOMMENDED ROADMAP

A practical path beyond the prototype

- 1 Stabilize the application boundary.** Move simulation state out of the global process variable, add per user sessions, structured logs, configuration files, and error monitoring.
- 2 Build a production service layer.** Add authentication, authorization, database persistence, a production WSGI server, deployment automation, and health checks.
- 3 Generalize the learned coordinator.** Choose either a checkpoint registry for supported shapes or a graph based policy that handles variable workers and orders. Retrain and reevaluate.
- 4 Add reproducible experiment management.** Record scenario configuration, seed, checkpoint version, code version, metrics, and artifacts for every run.
- 5 Create integration contracts.** Define safe inbound events and outbound recommendations before connecting any warehouse system.
- 6 Run human centered validation.** Evaluate usability, explanation quality, bias, workload, privacy, and appropriate human override mechanisms.
- 7 Introduce shadow mode.** Compare recommendations against real operations without allowing the system to control equipment or assign people.
- 8 Conduct a controlled pilot.** Proceed only after safety, security, legal, and statistical review.

Suggested priority

Phase	Focus	Exit condition
Phase 1	Reliability, session isolation, persistence, deployment	Multiple users can run independent reproducible episodes safely.
Phase 2	Variable shape policy and experiment registry	Every supported scenario has a trained, versioned, evaluated policy.
Phase 3	Integration simulator and shadow data	External events are ingested without controlling real operations.
Phase 4	Human factors, safety, security, and pilot	Formal approval and monitored pilot criteria are satisfied.

Recommended next technical milestone

Implement isolated persistent sessions and a checkpoint registry before adding more dashboard features. Those foundations remove the largest reliability and model compatibility constraints.

12. TECHNICAL REFERENCE

Project structure and ownership map

Path	Responsibility
<code>app.py</code>	Flask routes, session orchestration, dashboard HTML and CSS, explanations, serialization, fallback handling.
<code>env/collaborative_env.py</code>	Gymnasium environment, observations, legality, reward, stepping, termination, order creation.
<code>env/task_definitions.py</code>	Six task order template, dependencies, task times, priority, deadline, and slack model.
<code>env/workers.py</code>	AI worker behavior and common worker observation features.
<code>human/simulated_human.py</code>	Human behavior profiles, stochastic execution, training split, and held out split.
<code>agent/ppo_agent.py</code>	PPO network, action masking, rollout buffer, optimization, checkpoint save and load.
<code>agent/human_observer.py</code>	Rolling behavior features, enhanced observations, and visible observer snapshots.
<code>agent/curriculum.py</code>	Partner curriculum that emphasizes weak profiles while maintaining a sampling floor.
<code>baselines.py</code>	Random, Static, and Greedy coordination policies.
<code>utils/episode.py</code>	Single source of per episode counters.
<code>utils/metrics.py</code>	CES, aggregation, partner reports, and data model.
<code>utils/stats.py</code>	Bootstrap intervals and paired comparisons.
<code>train.py</code>	PPO training command line workflow.
<code>evaluate.py</code>	Evaluation across policies and partner profiles.
<code>ablation.py</code>	Ablation runs for observer, partner information, reassignment, and curriculum components.
<code>tests/test_dts_zsc.py</code>	Complete environment, model, statistics, baseline, and dashboard regression suite.
<code>checkpoints/</code>	Trained default PPO model and training log.
<code>results/</code>	Saved evaluation JSON, evidence JSON, episode data, and plots.
<code>docs/</code>	User guide, methodology documents, images, and this handoff guide.

Key commands

```
python app.py
python tests/test_dts_zsc.py
python evaluate.py --checkpoint checkpoints/agent_final.pt --baselines
python train.py --total_steps 250000 --seed 42
```

HANDOFF CHECKLIST

Acceptance and responsible use

Delivered files

Deliverable	Status	Location
Working source project	Verified	Project folder
Trained PPO checkpoint	Included	checkpoints/agent_final.pt
Saved evaluation and plots	Included	results/
Automated regression suite	41 passed	tests/test_dts_zsc.py
Interactive dashboard	HTTP 200	http://127.0.0.1:5000
Windows compatible share archive	Extraction passed	Desktop ZIP
Project handoff PDF	This document	Desktop PDF

Responsible use checklist

- Present results as simulation evidence only.
- Keep live episode demonstrations separate from saved repeated evaluations.
- Do not infer real employee ability, intent, health, or productivity from simulated profiles.
- Do not connect the prototype to operational equipment without safety engineering and approval.
- Do not expose the development server directly to untrusted networks.
- Label Greedy fallback behavior whenever PPO dimensions are incompatible.
- Retain code, checkpoint, configuration, and seed information with every published result.

Final status

The project is ready to share as a local research demonstration. It is not ready for production deployment or real warehouse decision making.

Prepared by _____

Reviewed by _____

Date _____

Version or release identifier _____